

Experiment 2: Cost-Aware Search over Complete Retry Rows

Detailed algorithm specification and evaluation protocol

COMS 6113 project

October 2, 2026

Executive summary

Experiment 2 asks whether we can find a strong **complete retry configuration** without directly testing every configuration on every benchmark question. Each configuration is a row such as

$$r = (m_0, m_1, m_2),$$

where m_0 is the model used on the original attempt and m_1, m_2 are the models used on the first and second retries. The selector must recommend one complete row. It may not recommend a partially tested row or treat a retry suffix as an independent arm.

The proposed structural method is **cross-fitted reach-aware pair racing (CRPR)**. CRPR is a candidate selector, in the same outer-search layer as Random Search, UCB, Bayesian Optimization, and Hill Climbing. It uses one-slot row similarity only to decide which complete rows to compare and how to allocate additional measurements. It never reuses a solver response, verifier state, or workflow prefix.

The project should first implement and measure a strong no-transfer control, called **cost-weighted common-random-number ranking and selection (CW-CRN-R&S)**, together with direct paired racing. CRPR is a transfer ablation until it demonstrates a saving at matched realized profiling spend. The components of CRPR are established ideas; the possible contribution is their tested adaptation to verifier-gated retry rows whose total charge is only known after execution.

This document is a design specification, not a claim that CRPR is novel or that it has already won on MathQA. The first SGFR replay was withdrawn after an implementation audit. A valid CRPR implementation and a fresh Experiment 2 trace are still pending.

1. Problem definition

1.1 Complete rows and questions

Let $\mathcal{M}$ be the fixed pool of solver models and let the retry limit be three attempts. The row bank is

$$\mathcal{R} = \mathcal{M}^3.$$

For the nine-model Experiment 1 pool, $|\mathcal{R}| = 9^3 = 729$. A paid profiling cell is one pair (r, q) : run the entire row r on question q using the deployment stopping rule, record the final workflow result, and charge every solver and verifier call that was reached. A cell is not a single model call.

For each cell we record:

- the answer-key-blind verifier result used during deployment;
- answer-key correctness, attached only for an offline labeled search or after the row is frozen for the final audit;
- the attempt at which the workflow stopped;

- whether each retry slot was reached;
- the input tokens and realized charge for every reached call;
- parse failures, timeouts, and other execution outcomes.

The verifier must not receive the MathQA answer key. The answer key belongs to evaluation, not to the deployment decision.

1.2 Cost

For a completed cell, the ledger is

$$C(r, q) = \sum_{s \in \text{reached solver slots}} c_{m_s} T_s(r, q) + c_v T_v(r, q),$$

where c_m is the registered model coefficient and T is the observed input-token count. The local pilot uses coefficient values as reproducible proxies; they are not API invoices. Output tokens, cache discounts, and latency are separate quantities unless a new experiment preregisters them.

Knowing the coefficient does **not** make $C(r, q)$ known before execution: the verifier can stop a row early, and that stopping decision depends on the response. Search budgets must therefore be compared using realized charge, with any whole-cell overshoot reported separately.

1.3 What the selector is allowed to see

The deployment-faithful track exposes the verifier signal and the cost ledger after each paid complete cell. Gold correctness stays hidden until selection confirmation or final audit. A labeled offline track may expose correctness on search questions as a diagnostic, but it must be labeled separately and cannot support a deployment claim by itself.

The final audit is never read while choosing a row, a budget, a threshold, a seed, or a model parameter.

2. Why similarity might help

Rows that differ in one ordered slot are connected in a Hamming graph. For example,

$$(A, B, B) \leftrightarrow (A, C, B)$$

is a one-slot neighbor. A shared question gives a paired comparison:

$$D_{a,b}(q) = Y_b(q) - Y_a(q),$$

where Y is the chosen quality signal. If the two rows behave similarly on a question, question difficulty cancels in the difference. This can make the difference easier to estimate than two independent accuracy means.

That observation is not a free prediction of an unmeasured row. The anchor row has to be paid for, the challenger still needs direct measurements when the comparison is uncertain, and the final candidates must be directly confirmed. Similarity is useful only if its reduction in uncertainty is large enough to compensate for the cost of measuring the anchor and the challenger.

The retry setting adds two complications:

1. later attempts are missing because an earlier verifier decision stopped the workflow;
2. the total charge is correlated with that same stopping behavior.

Therefore quality, retry reach, and charge must be modeled as separate signals. A row that looks similar in accuracy but reaches an expensive retry more often cannot be screened as equivalent.

3. CRPR algorithm

3.1 Registered blocks

Before observing outcomes, partition the search questions into four disjoint blocks:

1. **Calibration:** measure selected complete-row pairs and estimate their question-conditioned differences.
2. **Race:** measure incumbents directly and open challenger cells only under the registered rules below.
3. **Confirmation:** directly compare the surviving finalists on fresh questions.
4. **Final audit:** evaluate the one frozen row once, with answer-key labels attached only afterward.

Difficulty strata, question order, row order, edge panel, dollar caps, confidence level, non-inferiority margin, and exploration reserve are frozen before the first outcome. Natural early-stop and forced-retry questions are recorded as separate strata.

3.2 State kept by the selector

For every directly paid row/question cell, store:

- the complete-row quality signal;
- reach indicators for retry slots one and two;
- realized charge and its components;
- the question stratum and immutable cell identity.

For every opened neighbor comparison, store the paired residuals for quality, reach, and charge. The selector also maintains a current incumbent, a set of proposed challengers, confidence intervals, and the remaining profiling budget. A provisional score is never promoted to a final recommendation without direct confirmation.

3.3 Calibration

Select a complete incumbent and a one-slot neighbor, or use a pre-registered small edge panel. Run both complete rows on the calibration questions. For each stratum h , estimate:

$$\widehat{\Delta}_{a,b,h}^Y = \text{mean}_{q \in h}[Y_b(q) - Y_a(q)],$$

and corresponding differences in retry reach and realized charge. The calibration questions are selected before outcomes; an edge may not choose easy questions after seeing the incumbent.

The first implementation should use simple stratum means and bounded confidence intervals. A learned neural surrogate is unnecessary and would make leakage and calibration harder to audit.

3.4 Cross-fitted race

The calibration residual is applied only to a different race block. First pay the incumbent on a race question. Then:

- open the challenger if its predicted interval can still beat the incumbent;
- open it if the interval overlaps the decision boundary;
- open it when the direct-exploration reserve selects it;
- otherwise provisionally screen it, but do not call it the winner.

The prediction interval must include both calibration uncertainty and the uncertainty in the race-block incumbent. A point estimate alone is not a valid stopping rule after repeated adaptive looks.

Use separate gates for:

- quality difference;
- probability of reaching each retry slot;
- realized charge difference.

If calibration-versus-race drift exceeds the preregistered tolerance, disable residual transfer and fall back to direct paired racing. A rare expensive continuation reopens a gate even when the quality estimate looks favorable.

3.5 Cost-aware allocation

For each open comparison, estimate the expected reduction in the widest unresolved interval per unit of **new** complete-cell charge. The estimate uses only previously observed reach and charge data and includes a conservative uncertainty reserve. The allocator may not inspect the cost of an unpulled cell.

The policy has a global random or direct-testing reserve. This prevents a badly estimated neighborhood from trapping the search. When no edge passes the safety gate, the policy spends the reserve on direct complete-row tests.

3.6 Confirmation and recommendation

Every finalist is directly run on the confirmation block. The confirmation block is paid profiling data, not the final audit. Freeze the row only after the confirmation decision. The final audit then measures:

- answer-key correctness;
- verifier pass rate;
- retry reach;
- cold deployment charge.

The algorithm always returns a complete row. If the transfer model is not credible, the returned row comes from direct paired racing or the registered fallback selector.

3.7 Pseudocode

```
register rows, question blocks, strata, edge panel, costs, caps, and bounds
pay calibration cells for each registered incumbent/challenger pair
estimate quality, reach, and charge residuals by stratum
```

```
while profiling budget remains:
```

```
    choose incumbent or a global-reserve row
    propose a one-slot neighbor or a registered random challenger
    pay the incumbent on the next race question
    build cross-fitted quality/reach/charge intervals
```

```
    if challenger can beat, is unresolved, or reserve requires it:
```

```
        reserve worst-case charge
        pay the complete challenger cell
        update direct and paired records
```

```
    else:
```

```
        mark challenger provisionally screened
```

```
    if drift or covariance gate fails:
```

```
        disable transfer and use direct paired racing
```

```
directly evaluate every finalist on confirmation questions
```

```
freeze exactly one complete row
```

```
evaluate it on the untouched final audit
```

```
report accuracy, verifier behavior, reach, search spend, and cold cost
```

4. Required baselines and controls

CRPR should not be compared only with Random Search. The minimum comparison set is:

Method	Purpose
Random / uniform allocation	low-information reference
Matrix UCB-E	independent-arm upper-confidence allocation
Categorical Bayesian optimization	model-based configuration search
Arm elimination / hill climbing	classic adaptive search
Direct paired racing	same-question comparison without transfer
Cost-SySR or synchronized racing	established shared-question correlation baseline
CW-CRN-R&S	cost-aware correlated ranking with missing observations
C-LUCB-style control	correlated-arm conditional-bound baseline
SH-RR-style control	resource-rationed elimination baseline
CRPR	cross-fitted reach-aware transfer ablation

Falsification controls must include a shuffled row-slot graph, a question permutation control, unit-cost accounting, and permuted model costs. If CRPR benefits from a graph or pairing assumption, these controls should remove that benefit while preserving the rest of the experiment.

5. Experiment 2 protocol

Stage A: mechanics gate

Use three fixed model snapshots from the existing pool, giving 27 ordered rows. Use 200 search questions split into 60 calibration, 80 race, and 60 confirmation questions, plus 200 untouched audit questions. Use a sparse nine-edge panel covering every slot and model value. Register realized-dollar caps such as 10%, 20%, and 30% of the precomputed 27-row planning cost.

Stage A is a mechanics test, not publication evidence. It must automatically verify:

- the audit is inaccessible before row freeze;
- no unconfirmed row is selected;
- every complete cell has exact solver/verifier charge accounting;
- natural and forced-retry strata are labeled correctly;
- checkpoint resume identity binds the manifest and source code;
- transfer turns off after a failed drift check;
- whole-cell overshoot is reported rather than hidden.

Stage B: research run

After Stage A passes, use the nine-model pool and 729 complete rows. Register a fresh question bank, new final audit, fixed edge panel, method list, and absolute or formula-based dollar caps before outcomes. Compare methods at matched realized profiling spend, not merely at the same fraction parameter.

The primary endpoint at a registered budget b is the paired held-out accuracy difference:

$$\Delta A_b = A_b(\text{CRPR}) - A_b(\text{direct paired racing}),$$

alongside the spend difference

$$\Delta C_b = C_b(\text{direct paired racing}) - C_b(\text{CRPR}).$$

Choose the non-inferiority margin, error level, and minimum practical saving before opening the final block. Do not choose the most favorable budget after looking at the audit.

6. Reporting table

Every (selector, parameter setting, seed) should report:

- held-out final accuracy and paired confidence interval;
- verifier pass rate and false-pass/false-reject rates;
- retry reach by slot and by stratum;
- paid search cells and realized profiling charge;
- cold deployment charge of the recommended row;
- confirmation charge and any cap overshoot;
- whether the recommended row was directly observed before confirmation;
- time to reach each registered accuracy or regret level;
- the fraction of challenger cells screened by transfer;
- whether the transfer gate was disabled.

Search spend and cold deployment cost must remain separate. A cheaper search does not compensate for a worse final row unless a separate utility function was preregistered.

7. What would count as success

CRPR supports a positive result only if, at matched realized profiling spend:

1. its final-audit accuracy is non-inferior to direct paired racing;
2. it saves a preregistered practical amount of profiling spend or paid challenger cells;
3. its realized charge ledger is complete, including failed attempts and verifier calls;
4. its cold deployment cost is reported accurately;
5. the gain survives shuffled-graph, question-permutation, and unit-cost controls.

If it fails any of these conditions, the correct conclusion is that direct complete-row racing is safer for this setting. That negative result is useful: it would show that apparent row similarity does not produce a real profiling saving once retries, censoring, and cost are handled honestly.

8. Known limitations

The current Experiment 1 trace has only about 2.4% of workflows reaching a retry. This makes it a weak test of retry-aware structure. A forced-retry stratum is necessary to distinguish a genuinely unimportant retry slot from a slot that is simply rarely reached.

Two hundred final questions are pilot-scale. Near 38.5% accuracy, a simple 95% binomial interval has a half-width of roughly seven percentage points. Small apparent improvements must therefore be reported as exploratory unless the final audit is enlarged and the paired uncertainty is adequate.

The verifier in the existing pilot accepts many incorrect answers. A high verifier pass rate is not high answer accuracy. Future reports must show both signals and must not use verifier acceptance as a substitute for gold correctness.

9. Prior-art boundary and project status

Common-question comparisons, local-neighbor search, racing, correlated-arm best-arm identification, and resource-rationed elimination are established methods. CRPR should not be described as novel merely because it combines those words.

The possible project-specific intersection is narrower: complete ordered retry rows, verifier-censored reach, response-dependent whole-row charge, question-indexed paired measurements, and direct confirmation under a finite profiling budget. Whether that intersection is useful is an empirical question.

The earlier SGFR replay is withdrawn as evidence because its racing candidate list was empty and its same-block residual reduced to a directly measured candidate mean. CRPR is specified here but has not yet been

implemented or run on a fresh valid Experiment 2 trace. The existing repository has offline tests and prototypes, but the protocol-aware Experiment 2 runner remains the next implementation task.

10. Repository records

The design is derived from these tracked records:

- docs/research/deep-review/146-withdraw-sgfr-curve-and-freeze-e2-2026-09-30.md
- docs/research/deep-review/147-corrected-cross-fitted-racing-design-2026-09-30.md
- docs/research/deep-review/148-experiment2-decision-record-2026-09-30.md
- docs/research/deep-review/150-common-random-number-prior-art-correction-2026-09-30.md
- docs/research/deep-review/151-staged-experiment2-plan-2026-09-30.md
- docs/research/deep-review/152-runner-gap-for-experiment2-2026-09-30.md
- docs/research/deep-review/153-correlated-resource-baseline-audit-2026-09-30.md

The relevant prior-art families include common-random-number ranking and selection, SySRs, C-LUCB, resource-rationed elimination, F-Race/irace, ParamILS/SMAC, and the mentor’s cost-aware language-model evaluation work. These are comparison points and boundaries for the claim, not evidence that CRPR has already been validated.